



Using Bounded Estimation to Avoid Nonconvergence in Small Sample Structural Equation Modeling

Julie De Jonckere & Yves Rosseel

To cite this article: Julie De Jonckere & Yves Rosseel (2022) Using Bounded Estimation to Avoid Nonconvergence in Small Sample Structural Equation Modeling, Structural Equation Modeling: A Multidisciplinary Journal, 29:3, 412-427, DOI: [10.1080/10705511.2021.1982716](https://doi.org/10.1080/10705511.2021.1982716)

To link to this article: <https://doi.org/10.1080/10705511.2021.1982716>



Published online: 08 Feb 2022.



Submit your article to this journal [↗](#)



Article views: 542



View related articles [↗](#)



View Crossmark data [↗](#)



Citing articles: 1 View citing articles [↗](#)



Using Bounded Estimation to Avoid Nonconvergence in Small Sample Structural Equation Modeling

Julie De Jonckere and Yves Rosseel 

Ghent University

ABSTRACT

The most frustrating outcome of an SEM analysis is nonconvergence. Nonconvergence typically happens when the sample size is small ($N < 100$) or very small ($N < 50$). To minimize the frequency of nonconvergence, this paper proposes a solution called bounded estimation. The idea is to use data-driven lower and upper bounds for a subset of the model parameters during estimation. In this paper, we provide a rationale to compute these bounds, and we study the effect of different approaches to employ these bounds on the frequency of nonconvergence. A simulation study shows that bounded estimation dramatically decreases the frequency of nonconvergence in both correctly and misspecified models, without any (negative) effects on the quality of the point estimates for the unbounded parameters.

KEYWORDS

Small samples; estimation; nonconvergence

Introduction

As with other complex statistical models, it is widely believed that structural equation modeling (SEM) is best applied in settings where the sample size is rather large (Kline, 2010, p. 14). For example, Barrett (2007, p. 820) wrote that “SEM analyses based upon samples of less than 200 should simply be rejected outright for publication unless the population from which a sample is hypothesised to be drawn is itself small or restricted in size.” However, SEM is increasingly used in settings where the sample size is small ($N < 100$) or even very small ($N < 50$; Deng et al., 2018; Rosseel, 2020; Smid & Rosseel, 2020). There are often valid reasons for this. Perhaps the target population is indeed already small (for example: children with facial burn injuries), naturally resulting in small samples in practice. In other settings, collecting data may be either very time consuming or very expensive (for example: fMRI studies). In other words, despite the researcher’s best efforts, the final dataset may only contain a small number of observations. When SEM is then used to analyze this small dataset, several problems may occur. Apart from obvious lack of power, the point estimates may be biased, standard errors may be biased, and p -values for hypothesis tests or goodness-of-fit measures cannot be trusted. This has all been well documented in the literature (Bentler & Yuan, 1999; Nevitt & Hancock, 2004). But perhaps the most “visible” problem that may occur is that no solution can be found at all. We will refer to this outcome as “nonconvergence.” Loosely speaking, nonconvergence happens when the optimizer used in SEM software has failed to find a solution that satisfies a few criteria (Anderson & Gerbing, 1984; Boomsma, 1985). Typically, in this case, the SEM software will either not show any results at all, or it will show the solution from the last iteration which is deemed unsatisfactory, and indeed often contains nonsensical results (like a variance estimated to be, say, minus four hundred). If a researcher has invested a lot of time and energy to collect the data, and tries to

fit a well thought out model using SEM software, getting no (or only nonsensical) results may be a very frustrating experience. At this point, researchers may turn to suboptimal procedures, such as regression or path analysis based on sum scores. This is unfortunate, as these procedures often lead to biased results and misinformed conclusions (Cole & Preacher, 2014; Westfall & Yarkoni, 2016). Better solutions are available.

In statistics, there are two approaches that are often suggested when the sample size is small. The first approach is based on penalized likelihood methods (or regularization methods; Hastie et al., 2009). Penalized likelihood methods are similar to ordinary likelihood methods (like maximum likelihood estimation) but include an additional penalty term to control for the complexity of the model. The penalty term can be formulated to incorporate prior knowledge about the parameters or to discourage parameter values that are less realistic (e.g., far from zero). Although these penalization approaches have been around for a few decades, they have only recently been applied to SEM (Huang et al., 2017; Jacobucci et al., 2016). Unfortunately, a disadvantage of these penalized methods is that the user needs to indicate which parameters require penalization, and how much. In addition, SEM is a confirmatory approach, and the user needs to ensure that all parameters that are initially postulated in the model are not removed by the penalization. In addition, in our experience, penalized SEM still suffers from nonconvergence issues when the sample size is very small (say, $N < 50$).

A second solution is Bayesian estimation (B. Muthén & Asparouhov, 2012; Song & Lee, 2012). A major advantage of the Bayesian approach is that it does not rely on large sample asymptotics. Therefore, many small-sample problems from the frequentist framework (i.e., improper solutions, incorrect standard errors and test statistics) can be avoided. An essential ingredient of the Bayesian approach is the choice of priors. There is some evidence that informative priors are needed for

Bayesian SEM with small sample sizes (Smid & Rosseel, 2020). As the impact of the prior may be substantial when the sample size is small, it is recommended to always supplement a Bayesian SEM analysis with a sensitivity analysis, to assess the impact of the priors on the final results (Van Erp et al., 2018).

In this paper, we will propose a more simple solution. We will stick to the (unregularized) frequentist approach, as we believe this may be more familiar to many substantive researchers. In addition, our solution can be used with most (if not all) popular SEM software packages. We shall not claim that our solution is the “best” in some sense, but we do claim that it is effective and easy to use. As will be explained in detail in the next sections, we only exploit the ability of existing SEM software to impose simple upper and lower bounds on the estimation of model parameters. For a given parameter θ , we will compute a lower bound $\theta^{(l)}$ and an upper bound $\theta^{(u)}$ so that during estimation, the estimated value for θ stays within these bounds: $\theta^{(l)} \leq \hat{\theta} \leq \theta^{(u)}$. It turns out that if these bounds are chosen wisely, they can dramatically decrease the frequency of nonconvergence. A special case is to provide a zero lower bound (only) for error variances. It is well known that this helps to achieve convergence (Chen et al., 2001; Gerbing & Anderson, 1987), although the main motivation to keep the variance nonnegative was usually to avoid improper solutions, not to avoid nonconvergence. In our solution, we will propose lower and upper bounds for all model parameters, with the exception of regression coefficients in the structural part of the model. We call our solution “bounded estimation” and explain in a later section how it differs from “constrained estimation”.

The goal of this paper is to provide a rationale to find (data-driven) upper and lower bounds for several model parameters in an SEM and to study their effect on the frequency of convergence in both correctly and misspecified models. In addition, we will also study if these bounds impact the estimation of unbounded parameters. After all, the purpose of our solution is merely to help achieve convergence. We do not attempt to alter or improve the properties of the resulting point estimates. We just want to get (reasonable) point estimates. Furthermore, our solution is only valid for settings where the sample size (N) is larger than the number of observed variables (P).

If convergence can be obtained in small samples, then we get access to a variety of existing methods that can be used to improve the quality of the inference in small samples. Examples are the jackknife and the bootstrap (Efron & Tibshirani, 1993; Hesterberg, 2015). These methods are rarely used in the small sample SEM context because they were hampered by nonconvergence issues. Clearly, if the original sample did not converge, it is unlikely that a leave-one-out or a bootstrap sample of this sample will converge. A major motivation for our work is to open up the possibility to exploit the power of resampling methods in small samples. A minimum requirement for this is that the vast majority (if not all) of the jackknife or bootstrap samples result in a converged solution. As we will demonstrate in this paper, bounded estimation makes this possible.

The rest of the paper is organized as follows. First, we will discuss what we mean exactly with “nonconvergence” and provide a typical example. Then, we provide a rationale for choosing simple data-driven lower and upper bounds for the model parameters. Next, we study the frequency of nonconvergence for

several possible strategies. In a final section, we study the impact of bounded estimation on unbounded parameters. We conclude with a discussion of opportunities and limitations of our approach.

Nonconvergence in structural equation modeling

To illustrate the problem we try to address, consider the model in Figure 1.

We have generated a small dataset ($N = 20$) drawn from this population model. Next, we fitted this model using the package *lavaan* (Rosseel, 2012) in R (R Core Team, 2021) using the default maximum likelihood (ML) estimator. The R code we used can be found in Appendix A. For the sake of reproducibility, the values and eigenvalues of the variance–covariance matrix that were used for this analysis are listed in Appendix B. The variance–covariance matrix looks unremarkable, as do the eigenvalues. The *lavaan* output for this model and dataset can be found in Appendix C. The model did not converge, or in *lavaan* parlance, estimation “did not end normally” after a large number of iterations. At this point, it is important to stress a few facts. First, the model is correctly specified, and the data is drawn from a multivariate normal distribution, in accordance with the standard assumptions. Second, increasing the number of iterations does not help. Third, providing “better” starting values does not help either. And finally, as can be verified by the reader, the same model and dataset does not converge in the following SEM packages: *Mplus* 8.2 (L. K. Muthén & Muthén, 2010), *LISREL* 8.72 (Jöreskog & Sörbom, 1997) and *OpenMx* 2.19.1 (Neale et al., 2016). Interestingly, convergence is reached using *EQS* 6.1 (Bentler, 2004). We will explain later why this is the case. In all fairness, changing the seed or increasing the sample size may result in a different dataset that does lead to convergence. In a later section, we will see that for a sample of size $N = 20$, the percentage of converged solutions is about 70% across 10,000 replications for this model.

What does it mean if the model didn’t converge? In our example, we have used maximum likelihood estimation, and because SEMs are nonlinear models, ML estimation relies on an iterative optimization algorithm to obtain the final solution. Typical algorithms that are used in practice are: Newton-Raphson, Fisher scoring (or Gauss-Newton), and various quasi-Newton algorithms (Kochenderfer & Wheeler, 2019). In the case

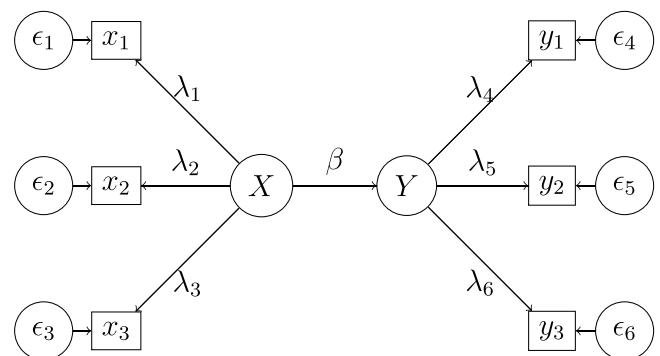


Figure 1. A simple model where the latent variable Y is regressed on the latent variable X . Each latent variable is measured by three observed indicators. The population values are as follows: $\lambda_1 = \lambda_4 = 1$, $\lambda_2 = \lambda_5 = 0.8$, $\lambda_3 = \lambda_6 = 0.6$, all (residual) variances are set to one, and the regression parameter (β) is set to 0.25.

of lavaan 0.6–9, optimization is (by default) carried out by the `nlminb()` function using a quasi-Newton approach. All these algorithms start with an initial guess for the model parameters (automatically generated or provided by the user). Then, using first and possibly second order derivatives, a new set of parameter values is obtained that increases the (log)likelihood of the data. This is a single iteration. Several iterations are typically needed to obtain the final solution. After each iteration, several “stopping criteria” are evaluated to check if the optimization algorithm can stop, or if another iteration is needed. These stopping criteria typically monitor the changes between the current iteration, and the previous iteration(s). These changes may be related to the discrepancy function that is used for the task at hand, or they may be related to the parameter values themselves (or both). If the changes are smaller than a certain threshold, the algorithm stops. But before we can claim “convergence,” we first need to check two additional criteria. If we use maximum likelihood, we must confirm that the solution is indeed a maximum. This requires that all elements of the gradient are (almost) zero, and that the Hessian is strictly negative definite. If all these conditions are satisfied, we have at least found a local maximum. But in our example, convergence was not reached. In fact, the message returned by the optimizer is “false convergence.” This means that at the last iteration, several attempts were made to improve the solution, but they all failed (because they all made the solution worse). This may happen if the stopping criteria are too strict (and therefore loosen them may solve the problem), but in our example, it is a sign that something is terribly wrong with the current set of parameter estimates. Indeed, inspection of the parameter estimates at the last iteration in [Appendix C](#) shows that the estimate for the residual variance of x_2 , and the estimate for the corresponding factor loading are completely out of range. It is a clear example of a nonconverged solution.

Now that we have identified the problematic parameters, there is an easy way to reach convergence. All we need to do is to inform the optimizer that the residual variance of x_2 should (ideally) be nonnegative. This means that we will impose a lower bound of zero for this parameter. Similarly, we will inform the optimizer that the factor loading for x_2 should not be larger than, say, 5.0. This is an upper bound for this parameter. [Appendix D](#) shows two different ways to implement these bounds in the lavaan model syntax. The first approach is to label the problematic parameters, and then explicitly state inequality constraints involving these labels. The second (recommended) approach is to use the `lower()` and `upper()` modifiers to specify the lower and upper bound of a parameter. Both approaches will produce the same result, and report that the model converged without any problems. However, close inspection of the output (see, [Appendix E](#)) reveals that the residual variance of x_2 is estimated to be zero. The `(lb)` label (for “lower bound”) in the output is a visual cue to alert the user that the estimated value for this parameter is equal to the lower bound that we have put in place. Think of these bounds as a fence: they prevent the optimizer to break through the fence and get lost in parameter space. Within these boundaries, zero was the “best” estimate the optimizer could find for this parameter for this particular dataset. This also explains why our problematic example did converge using the EQS software. By default, EQS does not allow for (error) variances to be negative. As a result, convergence was achieved.

In this paper, we will call the use of lower and upper bounds for the estimation of model parameters “bounded estimation”. We prefer not to use the term “constrained estimation.” We will reserve the latter term for settings where (equality and/or inequality) constraints are used to test specific hypotheses, leading the (constrained) models that are nested within unconstrained models. By contrast, “bounded estimation” should not impose any constraints on the model parameters other than keeping them within a reasonable range. Their only purpose is to help achieving convergence if the sample size is small. Ideally, if the sample size is large, these bounds should have no effect at all.

In our example, we used zero for the lower bound of the residual variance of x_2 . Zero seems a natural choice, as variances are supposed to be nonnegative. However, for the upper bound of the factor loading, we used the somewhat arbitrary value 5.0. Clearly, a better rationale is needed to choose this value. This is discussed in the next section.

Data-driven lower and upper bounds for SEM parameters

Given a model, and given a particular dataset, we will propose a simple method to determine the lower and upper bounds for most model parameters in an SEM. It is not our intention to choose the bounds that are the most strict from a theoretical perspective (see, for example, Ramsey & Gibson, 2006; Yanai & Ichikawa, 1990). On the contrary, we prefer fairly loose bounds that are perhaps too wide, as long as they help to achieve convergence. To compute the bounds, we will make use of the sample variances of the observed variables in the model. That is why the bounds are data-driven. All bounds are computed before optimization begins. In the section below, we first derive “standard” lower and upper bounds. In a next section, we will consider widening these standard bounds, in order to loosen the bounds even more.

Standard lower and upper bounds for the one-factor model

To fix ideas, consider a factor model with a single factor f and P observed indicators y_p , indexed by $p = 1, 2, \dots, P$. The one-factor model can be written as

$$y_p = \lambda_p f + \epsilon_p \quad p = 1, 2, \dots, P, \quad (1)$$

where λ_p is the factor loading for indicator p , and ϵ_p is the measurement error. We assume $E(\epsilon_p) = 0$ and $\text{Cov}(f, \epsilon_p) = 0$ for all p . For now, we will also assume that the measurement errors are uncorrelated, $\text{Cov}(\epsilon_p, \epsilon_{p'}) = 0$ for $p \neq p'$, although this assumption can easily be relaxed without consequences for our bounds. Taking the variance of both sides of (1), we obtain:

$$\text{Var}(y_p) = \lambda_p^2 \text{Var}(f) + \text{Var}(\epsilon_p). \quad (2)$$

This expression will guide us to find simple bounds for the factor loadings, the factor variance, and the residual variances of the indicators. Assuming that $\text{Var}(y_p)$ is strictly positive, we will choose the standard bounds in such a way, so that the sum of the two terms on the right is always nonnegative, but smaller than (or equal to) the term on the left. Variances have natural lower bounds of zero, so the standard lower bounds for $\text{Var}(\epsilon_p)$

and $\text{Var}(f)$ are zero. The standard upper bound for $\text{Var}(\epsilon_p)$ is equal to $\text{Var}(y_p)$, and is only attained if either λ_p or $\text{Var}(f)$ is zero. To obtain bounds for the factor loadings and the factor variance, we will make a distinction between two common approaches to fix the metric of the factor. In the first approach, the variance of the factor is fixed a priori to unity. In that case, the factor variance is no longer a free parameter, and no bounds are needed (or they could both be set to unity). If $\text{Var}(f) = 1$, then (2) simplifies to

$$\text{Var}(y_p) = \lambda_p^2 + \text{Var}(\epsilon_p). \quad (3)$$

Clearly, if $\text{Var}(\epsilon_p) = 0$, λ_p^2 can not be larger than $\text{Var}(y_p)$. Therefore, the standard upper bound for λ_p equals $\sqrt{\text{Var}(y_p)}$ while its lower bound equals $-\sqrt{\text{Var}(y_p)}$ (assuming we have no prior information about the sign of the factor loading). The bounds for this setting are summarized in Table 1.

The second approach sets the metric of the factor by fixing the first factor loading to unity: $\lambda_1 = 1$. We will call the corresponding indicator (y_1) the marker indicator. For this marker indicator, we have

$$\text{Var}(y_1) = \text{Var}(f) + \text{Var}(\epsilon_1). \quad (4)$$

It follows that the upper bound of $\text{Var}(f)$ is equal to $\text{Var}(y_1)$. For all other indicators $p \neq 1$, we have

$$\text{Var}(y_p) = \lambda_p^2 \text{Var}(f) + \text{Var}(\epsilon_p)$$

and we can not remove λ_p^2 from the equation. The upper bound for λ_p is now given by

$$\lambda_p^{(u)} = \sqrt{\frac{\text{Var}(y_p)}{\psi^{(l)}}}, \quad (5)$$

where we have used the notation $\psi^{(l)}$ for the lower bound of the factor variance. Clearly, if we define $\psi^{(l)} = 0$, we will not be able to put any finite bounds on λ_p (if $p \neq 1$). We propose the following solution. Assume that the reliability of the marker indicator (y_1) is strictly larger than zero: $\text{Rel}(y_1) > 0$. In that case, the upper bound of $\text{Var}(\epsilon_1)$ is no longer equal to $\text{Var}(y_1)$, but becomes $[1 - \text{Rel}(y_1)] \times \text{Var}(y_1)$. It follows from (4) that the lower bound for the factor variance is then equal to

$$\psi^{(l)} = \text{Var}(y_1) - [1 - \text{Rel}(y_1)] \times \text{Var}(y_1). \quad (6)$$

This lower bound will always be larger than zero if $\text{Var}(y_1) > 0$ and $\text{Rel}(y_1) > 0$. To illustrate, suppose $\text{Var}(y_1) = 10$, and the reliability of the marker indicator is very low, say $\text{Rel}(y_1) = 0.1$ (so 90% of y_1 is noise). Then, the upper bound of $\text{Var}(\epsilon_1)$ becomes $[1 - 0.1] \times 10 = 9$, and the lower bound for the factor variance is equal to $10 - 9 = 1$. Therefore, if the variance of another indicator p equals, say, $\text{Var}(y_p) = 16$, then the upper

Table 1. Standard lower and upper bounds for the factor variance, factor loadings and residual variances of a one-factor model, fixing the variance of the factor to unity for identification.

Parameter	Lower bound	Upper bound
$\text{Var}(f)$	1	1
λ_p	$-\sqrt{\text{Var}(y_p)}$	$\sqrt{\text{Var}(y_p)}$
$\text{Var}(\epsilon_p)$	0	$\text{Var}(y_p)$

Table 2. Standard lower and upper bounds for the factor variance, factor loadings and residual variances of a one-factor model, fixing the first factor loading to unity for identification. The lower bound for the factor variance $\psi^{(l)}$ is defined as $\psi^{(l)} = \text{Var}(y_1) - [1 - \text{Rel}(y_1)]\text{Var}(y_1)$, where $\text{Rel}(y_1)$ is a user-specified value for the reliability of the marker indicator y_1 .

Parameter	Lower bound	Upper bound
$\text{Var}(f)$	$\psi^{(l)}$	$\text{Var}(y_1)$
λ_1	1	1
λ_p	$-\sqrt{\text{Var}(y_p)/\psi^{(l)}}$	$\sqrt{\text{Var}(y_p)/\psi^{(l)}}$
$\text{Var}(\epsilon_p)$	0	$\text{Var}(y_p)$

bound for λ_p becomes $\sqrt{16/1} = 4.0$. If the reliability is 0.4, $\psi^{(l)}$ will be 4.0 and the upper bound for λ_p will be 2.0. If the reliability is 0.8, $\psi^{(l)}$ will be 8.0 and the upper bound for λ_p becomes 1.414214. Thus, the larger the assumed reliability for the marker indicator, the larger the lower bound of the factor variance, and the lower the upper bound of the factor loadings. In what follows, we will assume that $\text{Rel}(y_1) = 0.1$, unless otherwise indicated. The choice for 0.1 is very conservative as it assumes that the “best” (marker) indicator has at least a reliability of 0.1. Because the reliability of this marker indicator should be much higher in practice, this choice leads to fairly wide bounds (in particular for the factor loadings) that only help to achieve convergence, without affecting the quality of the parameter estimates. A summary of the upper and lower bounds in this setting is given in Table 2.

So far, we have assumed $\text{Cov}(\epsilon_p, \epsilon_{p'}) = 0$ for all $p \neq p'$. But if we allow this covariance to be free, we can easily find a lower and upper bound for this covariance, by using the formula of the correlation:

$$\text{Cor}(\epsilon_p, \epsilon_{p'}) = \frac{\text{Cov}(\epsilon_p, \epsilon_{p'})}{\sqrt{\text{Var}(y_p)}\sqrt{\text{Var}(y_{p'})}}. \quad (7)$$

Because the absolute value of a correlation cannot exceed a value of 1.0, we have the following inequality:

$$\sqrt{\text{Var}(y_p)}\sqrt{\text{Var}(y_{p'})} \leq |\text{Cov}(\epsilon_p, \epsilon_{p'})|, \quad (8)$$

where we have used $|x|$ to denote the absolute value of x . This immediately gives us an upper bound for the covariance. The lower bound is just the negative of this upper bound.

Standard lower and upper bounds for SEMs

We have derived our standard lower and upper bounds in the context of a one-factor model. However, we can use the same bounds for all the latent variables that are part of an SEM. If the model contains a structural part, we could also derive lower and upper bounds for the regression coefficients in the structural part. However, it turns out this is not only tedious to do so (Fuller, 1987, section 2.1), it is also unnecessary, provided we have used bounds for the (total) variances of the latent variables that are involved in the structural part. For all covariances in the model that are free parameters, we can use lower and upper bounds similar to (8). The final set of parameters that we need to discuss are the disturbances, or the residual variances of the endogenous (latent) variables in the structural part. Unless we have fixed a residual variance to unity, the standard

lower bound is again zero, and the standard upper bound is equal to $\text{Var}(y_1)$: the variance of the marker indicator for this latent variable. However, instead of using zero, lavaan uses 0.005 (per default) as the lower bound for residual latent variances.

Illustration: computing the bounds manually and automatically

Given a particular dataset, we can estimate these bounds by replacing $\text{Var}(y_p)$ by its sample estimate s_p^2 . In our example, these sample variances are on the diagonal of the variance-covariance matrix presented in [Appendix B](#). In the rest of the paper, we will always assume that the metric of a latent variable is set by fixing its first factor loading to unity. Therefore, we also need to make a (conservative) choice for the reliability of the marker indicator. By default, we use $\text{Rel}(y_1) = 0.1$. We now have all the ingredients to calculate the lower and upper bounds for all the model parameters of our model, with the exception of the regression coefficient (β).

We first compute the standard bounds manually. They are shown in the “lower” and “upper” columns of the first parameter table in [Appendix G](#). We start with the bounds for the residual variances of the observed variables. The lower bounds are set to zero, and the upper bounds are set to the observed sample variances (s_p^2) of the corresponding observed variables (lines 12–17). Only for the marker indicators do we replace the sample variance s_1^2 with $(1 - \text{Rel}) \times s_p^2$, where $\text{Rel} = 0.1$. That is why the upper bound for y_1 equals $(1 - 0.1) \times 2.15626340 = 1.940637$ and the upper bound for x_1 equals $(1 - 0.1) \times 1.412723 = 1.271451$ (line 12 and 15). The lower bound for the endogenous latent variable Y is set to 0.005 (line 18), while its upper bound is set to the sample variance of its marker variable (2.156). The lower bound for the exogenous latent variable X is obtained by the formula in (6):

$\psi_X^l = 1.412723 - (1 - 0.1) \times 1.412723 = 0.1412723$ (line 19). The upper bound is the sample variance 1.412723 of the marker indicator again. Finally, the upper bounds for the factor loadings are obtained from (5) (lines 5–10). For example, for y_2 , we find that $\sqrt{2.037233/\psi_Y^l} = 3.073757$ (line 6) where $\psi_Y^l = 2.15626340 - (1 - 0.1) \times 2.15626340 = 0.2156263$. The lower bounds are just the negative of the upper bounds.

By manually computing these bounds, they can be used in any SEM software package that allows to specify inequality constraints. In the lavaan package, we have created the `optim.bounds` argument to compute these standard bounds automatically. The R code is presented in [Appendix F](#). The `optim.bounds` argument contains (here) three elements. The first element (lower) is a vector of parameter classes for which we wish to compute automatic standard bounds. The `ov.var`, `lv.var` and `loadings` names refer to variances of observed variables, variances of latent variables, and factor loadings respectively. The second element (upper) is used to choose the parameters for which upper bounds need to be computed. The third element `min.reliability.marker` sets the minimum reliability that one is prepared to assume for the marker indicator for each latent variable. The default value is currently 0.1, but this may change in the future. Therefore, it is good practice to always

set it explicitly. Finally, the last element `min.var.lv.endo` sets the minimum value for the residual variance of endogenous latent variables. The current default is 0.005. It reflects the intuition that (in the presence of measurement error), it is unlikely that the residual variance of an endogenous variable in a regression is exactly zero. After a model has been fitted, the user can inspect the estimated bounds in the parameter table. A subset of the parameter table (including the estimated bounds) is presented in [Appendix G](#).

Fitting this model with the automatic (standard) bounds leads to the same output as the one presented in [Appendix E](#). Only the residual variance of x_2 is estimated at the (lower) boundary. All the other parameters are unaffected.

Widening the bounds

The standard bounds are straightforward to compute, and as we will see in the next section, they dramatically decrease the frequency of nonconvergence in settings where the sample size is small or even very small. However, it turns out that we can do slightly better by gently widening the bounds, as to give the optimizer somewhat more “wiggle room.” In some cases, this will increase the frequency of convergence even more (albeit with a very modest amount). However, our main motivation is to allow (again) for improper solutions. Without bounds, improper solutions may occur even if convergence is achieved. If we use the standard bounds, we may not only achieve convergence, we may also (unknowingly) change an improper solution to an proper solution. This is in conflict with our main purpose for using bounded estimation: only to help achieve convergence, but nothing more. Therefore, we will explore the option to widen the bounds. To proceed, define the range of a bounded parameter as the difference (or distance) between its upper and lower bound. As an example, suppose the upper bound of a parameter is +5, while its lower bound is -5. The range is $(+5 - (-5)) = 10$. Suppose we wish to widen the range with 10%. Then, the upper and lower bounds become +6 and -6, respectively. We could also choose to only change the upper bound and keep the lower bound, or vice versa. Of course, if we widen the bounds too much, we may loose their ability to achieve convergence. After much experimentation, we found that a good compromise is to increase the upper bounds for the observed variances, latent variances, and factor loadings with 20%, 30%, and 10%, respectively, of the standard range. For the lower bounds, we increase the range of the observed variances and factor loadings with 5% and 10%, respectively, of the standard range. Note that we do not lower the lower bounds of the latent variances. We keep them strictly nonnegative. But we do allow for (residual) variances of indicators to be (slightly) negative. If this happens, most SEM software packages (including lavaan) will issue a warning, alerting the user that an improper solution has been found. This is not always a reason for concern, but we feel it is important that the user is informed about this.

To illustrate how to achieve these widened bounds using lavaan, see the second model fit in [Appendix F](#). The `optim.bounds` list now contains two additional elements: `lower.factor` and `upper.factor` that can be used to specify the amount of widening (as a factor of the total range) that should be applied

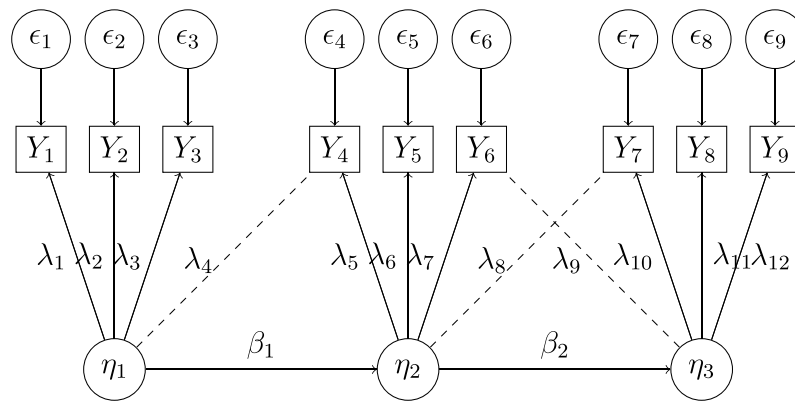


Figure 2. The population model used for study 2. The population value for β_1 and β_2 is 0.6. For the factor loadings, the population values are fixed at 1.0, except for the crossloadings (λ_4 , λ_8 , λ_9) which have a population value of 0.3. The latent (residual) variances are set to 1.0. The residual variances of the indicators are 0.51 for the y_1 , y_2 , y_3 , y_5 , y_8 , and y_9 indicators, and 0.2895 for the y_4 , y_6 , and y_7 indicators.

to the lower and upper bounds respectively. The resulting parameter table (including the bounds and the estimated parameters) is shown at the bottom half of [Appendix G](#).

The impact of bounded estimation on convergence

To assess the impact of bounded estimation on convergence, we report the results of two studies.¹ In the first study, we used the population model in [Figure 1](#) and fitted a correctly specified model. In the second study, we used the population model in [Figure 2](#) and fitted a variety of models with an increasing number of misspecifications. In both studies, we considered four different approaches to determine the bounds:

- **no-bounds:** this is just vanilla maximum likelihood estimation without any bounds;
- **ov.var:** zero lower bounds are used for the (residual) variances of the observed variables;
- **standard:** standard lower and upper bounds are used as described in [Table 2](#); we set `min.reliability.marker` to 0.1 and `min.var.lv.endo` to 0.005;
- **wide:** same settings as the standard bounds, but upper bounds for observed variances, latent variances and factor loadings are increased with 20%, 30%, and 10% of the standard range, respectively; lower bounds for observed variances and factor loadings are decreased with 5% and 10% of the standard range, respectively.

We used 12 different sample sizes: 10, 15, 20, 25, 30, 40, 50, 60, 70, 80, 90, 100. For each sample size, 10,000 datasets were generated. For each replication, we draw a random sample of size N from a multivariate normal distribution with a zero mean vector and a variance–covariance matrix that corresponds to the model-implied covariance matrix of the population model. For each replication, lavaan 0.6–9 was used to fit a model using one of the four approaches to set the bounds. For each fit, we observed if the estimation procedure led to convergence or not. Convergence is only reached if three conditions are met: 1) the optimizer has ended normally, 2) the additional gradient check (verifying that all elements are near

zero) is positive, and 3) not all the standard errors are NA (an indirect check that the information matrix is not singular). If the model converged, we saved the values of the estimated parameters for later use.

Study 1: Correctly specified model

In this study, we used the population model in [Figure 1](#) and always fitted a correctly specified model. This model has 13 free parameters. Simple starting values were used where all factor loadings and variances were set to 1.0, while the regression coefficient was set to zero. The number of replications that did not converge per sample size is presented in [Table 3](#).

Unsurprisingly, if no bounds are used, nonconvergence occurred very frequently. For the smallest sample size ($N = 10$), nonconvergence occurred in about 51% of the replications. But even when $N = 50$, roughly 5% of the replications still did not converge. For $N = 100$, this drops to about 0.21%. The “ov.var” approach performs much better. When $N = 10$, only 9.21% of the replications did not converge. This percentage drops rapidly to zero from $N = 70$ onwards. The standard approach performs even better, in particular for the very small sample sizes ($N < 50$). If $N > 50$, there are a few replications that do not converge, while they do converge in the ov.var approach. In those cases, the additional bounds, and in particular the upper bounds have prevented convergence in those replications. Finally, with one exception, the wide approach always converged for this model.

Study 2: Varying degrees of misspecifications

In this study, we use the population model in [Figure 2](#). We borrowed this model from Chen et al. (2001) (see, their [Figure 1](#)). This model allows us to fit four models with a varying degree of misspecification. In the first model (M0), all cross-loadings were included, and the fitted model was correct. In the second model (M1), one cross-loading (λ_8) was omitted. In the third model (M2), two cross-loadings (λ_8 and λ_9) were omitted, while in the fourth model (M4), three cross-loadings were omitted (λ_8 , λ_9 and λ_4). For each replication, we fitted these four models, using the four different approaches to set the bounds.

¹The code that was used for these two studies can be found using the following link: <https://osf.io/f7z6j/>

Table 3. Number of nonconverged replications (out of 10,000) for a correctly specified model based on the population model of Figure 1.

Sample size	no-bounds	ov.var	Standard	Wide
10	5115	921	13	0
15	3922	414	13	1
20	2973	233	14	0
25	2240	135	11	0
30	1679	66	5	0
40	934	27	4	0
50	522	5	4	0
60	264	1	8	0
70	167	0	5	0
80	73	0	2	0
90	37	0	2	0
100	21	0	1	0

Table 4. Number of nonconverged replications (out of 10,000) for four different models (M0, M1, M2, and M3) based on the population model of Figure 2.

Model	Sample size	no-bounds	ov.var	Standard	Wide
M0	10	6850	2672	50	40
	15	4139	1474	10	15
	20	2626	876	1	6
	25	1589	523	1	5
	30	1117	337	2	0
M1	10	5963	1990	33	20
	15	3147	934	9	6
	20	1728	516	6	1
	25	932	265	4	1
	30	628	185	2	1
M2	10	4886	1649	25	7
	15	2183	789	4	0
	20	1079	462	1	0
	25	488	236	0	0
	30	313	164	1	0
M3	10	3493	873	11	0
	15	1272	249	4	1
	20	489	87	2	0
	25	201	27	1	0
	30	115	14	0	0

The population values were used as starting values. The number of replications that did not converge per sample size is presented in Table 4. We only show the results for the smallest sample sizes (up to $N = 30$), as the general patterns are already clear.

A first observation is that frequencies of nonconvergence for the correctly specified model (M0) are higher compared to those of Study 1. This is to be expected, as the model is more complex, and contains more (23) free parameters. When $N = 10$, about 68.5% of the replications did not converge. Not shown in the table, this percentage drops to 2.21% when $N = 50$, and 0.09% when $N = 100$. The “ov.var” approach performs better but still results in 26.72% of nonconverged replications when $N = 10$. Not shown in the table, this percentage drops to 0.45% when $N = 50$, and 0.01% when $N = 100$. In comparison, the percentages of nonconvergence for the standard and wide approaches are extremely low. When $N = 10$, the percentages are 0.5% and 0.4%, respectively, and drop below 0.1% from $N = 20$ onwards. For both approaches, they become zero from $N = 60$ onwards. Another clear pattern that can be observed in Table 4 is that the frequencies of nonconvergence decrease when the number of misspecifications increase. This may seem counter-intuitive at first, but the misspecified models contain fewer free parameters. The number of free parameters are 23, 22, 21, and 20 for the M0, M1, M2, and M3

models, respectively. It would seem that models with fewer free parameters are more likely to converge, regardless of whether the model is specified correctly or not. A last observation is that the wide approach is usually better than the standard approach (in terms of convergence) and usually reaches 0% of nonconvergence from $N = 30$ onwards.

Finally, we examined the only replication that did not converge for model M3 while using the wide approach. It turns out that for this particular case, decreasing the lower bound for the observed variances with 10% (instead of 5%) does lead to convergence. In fact, for most specific cases that did not converge with the current settings for the wide approach, it is possible to find a set of alternative settings that will lead to convergence. The default settings that we have used in this paper are based on a compromise that seems to work well in most settings, but applied users can always tweak the settings if convergence is not achieved automatically.

The impact of bounded estimation on unbounded parameters

The standard and wide approach successfully increase the frequency of converged solutions, even in very small samples. However, these bounds are not supposed to introduce additional bias. Therefore, in this section, we investigate the impact of bounded estimation on the point estimate of the only unbounded parameter in the model of Figure 1: the structural parameter β . In the simulation study that we reported in the previous section, we always saved the estimated parameter vector. From this, we can extract $\hat{\beta}$, the estimate for the regression coefficient. Based on all converged replications, we computed the means and the standard deviations for $\hat{\beta}$. They are presented in Table 5. Because the distribution of $\hat{\beta}$ is skewed, we also report the trimmed means (where 0.1% of the most extreme values are removed before we take the mean), and the median values in Table 6. The population value for β is equal to 0.25. Based on this, we report the relative mean bias and mean squared error (multiplied by 100) in Table 7. It is important to realize that the values are only reported for converged replications. Therefore, the mean estimate of 0.47 for $N = 10$ in the no-bounds column is based on (only) 4,885 replications, while the mean estimate of 0.28 in the wide column is based on 10,000 replications. But if we would only report the values for those replications for which the no-bounds method converged, this would not change the patterns in these tables.

We observe that the maximum likelihood estimate of β (without any bounds) is positively biased, up to 88% if $N = 10$. At least if we look at the means. The mean bias decreases rapidly and is almost negligible for sample sizes larger than $N = 60$. A similar pattern can be observed for the ov.var approach, although the highest bias is only 45% if $N = 15$. By contrast, both the standard and wide approach show much less bias (less than 20%) in the smallest sample sizes, and almost no bias (less than 10%) if the sample size is larger than $N = 30$. However, we have to be careful with these results, as the distribution of $\hat{\beta}$ is skewed. The median values show almost no bias, even in the no-bounds approach. The trimmed means closely follow the raw

Table 5. The mean and standard deviations (between the parentheses) of the estimated values for β , the unbounded regression coefficient in the model. The population value is $\beta = 0.25$.

Sample size	no-bounds	ov.var	Standard	Wide
10	0.47 (2.11)	0.34 (2.39)	0.29 (1.04)	0.28 (1.04)
15	0.42 (1.89)	0.36 (1.61)	0.29 (0.78)	0.29 (0.79)
20	0.39 (1.43)	0.33 (1.43)	0.29 (0.65)	0.30 (0.65)
25	0.34 (0.90)	0.31 (0.97)	0.29 (0.55)	0.29 (0.55)
30	0.34 (1.23)	0.31 (0.99)	0.29 (0.48)	0.28 (0.48)
40	0.29 (0.66)	0.28 (0.69)	0.27 (0.37)	0.27 (0.37)
50	0.28 (0.42)	0.28 (0.37)	0.27 (0.31)	0.27 (0.31)
60	0.27 (0.37)	0.27 (0.37)	0.27 (0.28)	0.27 (0.28)
70	0.26 (0.25)	0.26 (0.24)	0.26 (0.24)	0.26 (0.24)
80	0.26 (0.22)	0.26 (0.22)	0.26 (0.22)	0.26 (0.22)
90	0.26 (0.20)	0.26 (0.20)	0.26 (0.20)	0.26 (0.20)
100	0.26 (0.19)	0.26 (0.18)	0.26 (0.18)	0.26 (0.18)

Table 6. The trimmed means (where 0.1% of the most extreme values are removed) and median values (between the parentheses) for β .

Sample size	no-bounds	ov.var	Standard	Wide
10	0.44 (0.28)	0.32 (0.25)	0.29 (0.26)	0.29 (0.25)
15	0.39 (0.27)	0.32 (0.24)	0.29 (0.25)	0.28 (0.24)
20	0.36 (0.26)	0.32 (0.25)	0.29 (0.25)	0.29 (0.25)
25	0.33 (0.24)	0.30 (0.24)	0.29 (0.24)	0.28 (0.24)
30	0.31 (0.26)	0.29 (0.25)	0.28 (0.25)	0.28 (0.24)
40	0.28 (0.24)	0.27 (0.24)	0.27 (0.24)	0.27 (0.24)
50	0.28 (0.25)	0.27 (0.25)	0.27 (0.25)	0.27 (0.24)
60	0.27 (0.25)	0.27 (0.24)	0.27 (0.24)	0.26 (0.24)
70	0.26 (0.24)	0.26 (0.24)	0.26 (0.24)	0.26 (0.24)
80	0.26 (0.24)	0.26 (0.24)	0.26 (0.24)	0.26 (0.24)
90	0.26 (0.24)	0.26 (0.24)	0.26 (0.24)	0.26 (0.24)
100	0.25 (0.24)	0.25 (0.24)	0.25 (0.24)	0.25 (0.24)

Table 7. The relative mean bias (in percent2es) and mean squared error (multiplied by 100 between the parentheses) for β .

Sample size	no-bounds	ov.var	Standard	Wide
10	88 (451)	34 (570)	15 (108)	14 (107)
15	69 (359)	45 (259)	15 (61)	14 (62)
20	55 (205)	33 (206)	18 (43)	18 (43)
25	36 (82)	26 (94)	15 (30)	15 (30)
30	36 (151)	23 (99)	14 (23)	14 (23)
40	18 (44)	12 (48)	9 (14)	9 (14)
50	12 (18)	11 (14)	9 (10)	9 (10)
60	10 (13)	9 (13)	7 (8)	7 (8)
70	6 (6)	5 (6)	5 (6)	5 (6)
80	6 (5)	6 (5)	6 (5)	6 (5)
90	4 (4)	4 (4)	4 (4)	4 (4)
100	2 (3)	2 (3)	2 (3)	2 (3)

means, suggesting that the skewness is not merely due to a few outliers. But most importantly, the standard and wide approach do not increase the bias. And this is what we wished to verify. The standard deviations and mean squared errors show another interesting pattern. The variability is very high for the no-bounds and ov.var approach when the sample size is very small. This variability is significantly lowered in the standard and wide approach. Finally, there is (almost) no difference between the standard and wide approach. Both the (relative) mean (or median) bias are very similar, as is the variability.

As for the other (bounded) parameters, the pattern is always the same: The (relative) mean bias is high for the very small sample sizes in the no-bounds and (to a lesser extent) in the ov.var approach, while the bias is always lower in the standard and wide approaches, compared to the no-bounds approach. The bias is always upwards for factor loadings and downwards for

variances. From $N = 70$ onwards, the bias is negligible. In conclusion, bounded estimation would seem to have only beneficial effects on bias, both for bounded and unbounded parameters.

Discussion

Nonconvergence is perhaps the most frustrating outcome of an SEM analysis. Getting no results at all is a nightmare for many substantive researchers who often spend a lot of effort to collect the data. Many of them abandon SEM altogether and turn to suboptimal procedures that ignore the measurement error. Still others try their luck with Bayesian approaches only to discover that this requires a new set of skills. Yet, the solution is rather simple: putting lower and upper bounds on (a subset of) the model parameters can dramatically increase the frequency of convergence. In this paper, we provided a rationale to compute these bounds and suggested two approaches: the standard approach and the wide approach. In terms of convergence, both approaches reduce the number of nonconverged solutions to nearly zero, without (negatively) affecting the bias of the estimated parameters. We recommend users to always use these bounds, in particular in settings where $N < 50$. Using these bounds has no negative effects on the quality of the estimates of unbounded parameters. The only side-effect is that the variability of the point estimates is much smaller across replications, in particular when the sample size is very small. This leads to smaller mean squared errors. Overall, bounded estimation seems to be both effective and harmless.

Of course, if a model converges, this does not take away all the other problems that arise if the sample size is small. Improper solutions may still occur, point estimates (in particular of variances) may be biased, and hypothesis tests may lack power. In other words, researchers should always try to collect as much data as possible and not rely on bounded estimation as an excuse to collect less data.

A potential advantage of our bounded estimation procedure is that it unlocks the possibility to use resampling methods to get (for example) improved standard errors in small samples. As a proof of concept, we fitted our example model (with $N = 20$) with wide bounds and requested bootstrap-based standard errors. The R code can be found in [Appendix H](#). A thousand bootstrap samples were generated, and all of them converged. The output is shown in [Appendix I](#). Of course, future research is needed to evaluate the quality of these bootstrap-based standard errors, in particular when the sample size is very small. But at the very least, bounded estimation makes it possible to study these bootstrap-based standard errors in the small sample SEM context. Another advantage is that the bounds work with any estimator (not just maximum likelihood) and so can also be applied in settings where we have missing data, categorical data, or multilevel data.

There is one important limitation to our approach: the automatic way to generate bounds (using the `optim.bounds` argument) as currently implemented in lavaan 0.6–9 does not work if the model also contains *linear* equality constraints. The reason is that lavaan then projects the set of free (but constrained) parameters to a lower-dimensional set of unconstrained parameters (see, Nocedal & Wright, 2006, p. 428). However, the bounds no longer have any meaning in this

lower-dimensional parameter space. Adjusting the bounds in the presence of linear equality constraints is far from trivial (see, Nocedal & Wright, 2006, p. 434) and is deferred to future work. An alternative is to compute the lower and upper bounds manually and use the general method using explicit inequality constraints as illustrated in the upper half of Appendix D.

in the context of structural equation modeling. Future research is needed to investigate if bounded estimation has similar benefits when applied to other complex statistical models that rely on iterative optimization.

As a final note, when no bounds are used, and the estimation procedure does not converge for a particular dataset, we still do not fully understand why that is the case. We can blame the (small) sample size, but other datasets with exactly the same sample size often converge without any problem. Given a dataset, and given a model, we can not (yet) predict (with high accuracy) if the estimation procedure will converge or not. We are currently investigating this intriguing problem and will report our findings in another paper.

Disclosure statement

No potential conflict of interest was reported by the author(s).

ORCID

Yves Rosseel  <http://orcid.org/0000-0002-4129-4477>

References

- Anderson, J. C., & Gerbing, D. W. (1984). The effect of sampling error on convergence, improper solutions, and goodness-of-fit indices for maximum likelihood confirmatory factor analysis. *Psychometrika*, 49, 155–173. <https://doi.org/10.1007/BF02294170>
- Barrett, P. (2007). Structural equation modelling: Adjudging model fit. *Personality and Individual Differences*, 42, 815–824. <https://doi.org/10.1016/j.paid.2006.09.018>
- Bentler, P. M., & Yuan, K. H. (1999). Structural equation modeling with small samples: Test statistics. *Multivariate Behavioral Research*, 34, 181–197. <https://doi.org/10.1207/S15327906Mb340203>
- Bentler, P. M. (2004). *EQS 6 structural equations program book* [Computer software manual]. Multivariate Software, Inc.
- Boomsma, A. (1985). Nonconvergence, improper solutions, and starting values in LISREL maximum likelihood estimation. *Psychometrika*, 50, 229–242. <https://doi.org/10.1007/BF02294248>
- Chen, F., Bollen, K. A., Paxton, P., Curran, P. J., & Kirby, J. B. (2001). Improper solutions in structural equation models: Causes, consequences, and strategies. *Sociological Methods & Research*, 29, 468–508. <https://doi.org/10.1177/0049124101029004003>
- Cole, D. A., & Preacher, K. J. (2014). Manifest variable path analysis: Potentially serious and misleading consequences due to uncorrected measurement error. *Psychological Methods*, 19, 300. <https://doi.org/10.1037/a0033805>
- Deng, L., Yang, M., & Marcoulides, K. M. (2018). Structural equation modeling with many variables: A systematic review of issues and developments. *Frontiers in Psychology*, 9, 580. <https://www.frontiersin.org/article/10.3389/fpsyg.2018.00580>
- Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.
- Fuller, W. A. (1987). *Measurement error models*. John Wiley & Sons.
- Gerbing, D. W., & Anderson, J. C. (1987). Improper solutions in the analysis of covariance structures: Their interpretability and a comparison of alternate respecifications. *Psychometrika*, 52, 99–111. <https://doi.org/10.1007/BF02293958>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction, second edition*. Springer.
- Hesterberg, T. C. (2015). What teachers should know about the bootstrap: Resampling in the undergraduate statistics curriculum. *The American Statistician*, 69, 371–386. <https://doi.org/10.1080/00031305.2015.1089789>
- Huang, P.-H., Chen, H., & Weng, L.-J. (2017). A penalized likelihood method for structural equation modeling. *Psychometrika*, 82, 329–354. <https://doi.org/10.1007/s11336-017-9566-9>
- Jacobucci, R., Grimm, K. J., & McArdle, J. J. (2016). Regularized structural equation modeling. *Structural Equation Modeling*, 23, 555–566. <https://doi.org/10.1080/10705511.2016.1154793>
- Jöreskog, K. G., & Sörbom, D. (1997). *LISREL 8: User's reference guide* [Computer software manual]. Scientific Software International.
- Kline, R. B. (2010). *Principles and practice of structural equation modeling, fourth edition*. The Guilford Press.
- Kochenderfer, M. J., & Wheeler, T. A. (2019). *Algorithms for optimization*. The MIT Press.
- Muthén, B., & Asparouhov, T. (2012). Bayesian structural equation modeling: A more flexible representation of substantive theory. *Psychological Methods*, 17, 313. <https://doi.org/10.1037/a0026802>
- Muthén, L. K., & Muthén, B. O. (2010). *Mplus user's guide* (6th ed.). [Computer software manual]. Muthén & Muthén.
- Neale, M. C., Hunter, M. D., Pritikin, J. N., Zahery, M., Brick, T. R., Kirkpatrick, R. M., Estabrook, R., Bates, T. C., Maes, H. H., & Boker, S. M. (2016). OpenMx 2.0: Extended structural equation and statistical modeling. *Psychometrika*, 81, 535–549. <https://doi.org/10.1007/s11336-014-9435-8>
- Nevitt, J., & Hancock, G. R. (2004). Evaluating small sample approaches for model test statistics in structural equation modeling. *Multivariate Behavioral Research*, 39, 439–478. http://dx.doi.org/10.1207/S15327906MBR3903_3
- Nocedal, J., & Wright, S. (2006). *Numerical optimization*. Springer Science & Business Media.
- R Core Team. (2021). *R: A language and environment for statistical computing* [Computer software manual]. <https://www.R-project.org/>
- Ramsey, P. H., & Gibson, W. (2006). Improved communality estimation in factor analysis. *Journal of Statistical Computation and Simulation*, 76, 93–101. <https://doi.org/10.1080/00949650412331320855>
- Rosseel, Y. (2012). lavaan: An R package for structural equation modeling. *Journal of Statistical Software*, 48, 1–36. <https://doi.org/10.18637/jss.v048.i02>
- Rosseel, Y. (2020). Small sample solutions for structural equation modeling. In R. van de Schoot & M. Miočević (Eds.), *Small sample size solutions* (pp. 226–238). Routledge.
- Smid, S. C., & Rosseel, Y. (2020). Sem with small samples: Two-step modeling and factor score regression versus Bayesian estimation with informative priors. In R. van de Schoot & M. Miočević (Eds.), *Small sample size solutions* (pp. 239–254). Routledge.
- Song, X.-Y., & Lee, S.-Y. (2012). *Basic and advanced Bayesian structural equation modeling: With applications in the medical and behavioral sciences*. John Wiley & Sons.
- Van Erp, S., Mulder, J., & Oberski, D. L. (2018). Prior sensitivity analysis in default Bayesian structural equation modeling. *Psychological Methods*, 23, 363. <https://doi.org/10.1037/met0000162>
- Westfall, J., & Yarkoni, T. (2016). Statistically controlling for confounding constructs is harder than you think. *PLoS ONE*, 11, e0152719. <https://doi.org/10.1371/journal.pone.0152719>
- Yanai, H., & Ichikawa, M. (1990). New lower and upper bounds for communality in factor analysis. *Psychometrika*, 55, 405–409. <https://doi.org/10.1007/BF02295295>

Appendix A

The motivating example: R code

```

1 library("lavaan")
2
3 # define the population model
4 pop.model <- '
5   # factor loadings
6   Y =~ 1*y1 + 0.8*y2 + 0.6*y3
7   X =~ 1*x1 + 0.8*x2 + 0.6*x3
8
9   # regression part
10  Y ~ 0.25*X
11  '
12
13 # set seed
14 set.seed(8)
15
16 # generate random sample (N=20)
17 Data <- simulateData(pop.model, sample.nobs = 20L)
18
19 # define the model to be fitted
20 model <- '
21   Y =~ y1 + y2 + y3
22   X =~ x1 + x2 + x3
23   Y ~ X
24  '
25
26 # fit sem using ML (without bounds)
27 fit <- sem(model, data = Data, estimator = "ML",
28           optim.attempts = 1L, verbose = TRUE)
29 summary(fit)

```

Appendix B

The motivating example: sample statistics

```

1 # sample variance-covariance matrix
2 > COV <- lavInspect(fit, "samplestats", add.class = FALSE)$cov
3 > COV
4
5           y1           y2           y3           x1           x2           x3
6 y1  2.15626340  0.81960962  0.51932004  0.11309024  0.14229266 -0.04280304
7 y2  0.81960962  2.03723315  0.74608735  0.33553484  0.04825119  0.27927525
8 y3  0.51932004  0.74608735  1.56276168  0.02812082 -0.39674173 -0.20118422
9 x1  0.11309024  0.33553484  0.02812082  1.41272347  1.38072174  0.14441356
10 x2  0.14229266  0.04825119 -0.39674173  1.38072174  2.36195654  0.70487174
11 x3 -0.04280304  0.27927525 -0.20118422  0.14441356  0.70487174  1.55238559
12
13 # eigen values
14 > eigen(COV, symmetric = TRUE, only.values = TRUE)$values
15 [1] 3.7054583 3.3378688 1.5233660 1.3846538 0.8604623 0.2715146

```

Appendix C

The motivating example: lavaan output

```

1  lavaan 0.6-9 did NOT end normally after 4464 iterations
2  ** WARNING ** Estimates below are most likely unreliable
3
4  Estimator                      ML
5  Optimization method            NLMINB
6  Number of model parameters      13
7
8  Number of observations          20
9
10 Model Test User Model:
11
12 Test statistic                  NA
13 Degrees of freedom              NA
14
15 Parameter Estimates:
16
17 Standard errors                Standard
18 Information                    Expected
19 Information saturated (h1) model Structured
20
21 Latent Variables:
22
23      Estimate Std.Err z-value P(>|z|)
24 Y =~
25   y1          1.000
26   y2          1.683      NA
27   y3          1.051      NA
28 X =~
29   x1          1.000
30   x2        302.417      NA
31   x3          0.428      NA
32
33 Regressions:
34
35      Estimate Std.Err z-value P(>|z|)
36 Y ~
37   X          -0.159      NA
38
39 Variances:
40
41      Estimate Std.Err z-value P(>|z|)
42 .y1          1.706      NA
43 .y2          0.763      NA
44 .y3          1.066      NA
45 .x1          1.408      NA
46 .x2        -415.125      NA
47 .x3          1.552      NA
48 .Y           0.450      NA
49 X           0.005      NA

```


Appendix D

```

1 # adding two bounds:
2 # - lower (zero) bound for x2 ~~ x2
3 # - upper (5) bound for Y == x2
4
5 # 1. general method — using labels and inequality constraints
6 model1 <- '
7     Y == y1 + y2 + y3
8     X == x1 + 15*x2 + x3
9     Y ~ X
10    x2 ~~ rvar2*x2
11
12    # imposing lower and upper bounds
13    15 < 5
14    rvar2 > 0
15    '
16
17 # fit model using nlminb.contr()
18 fit1 <- sem(model1, data = Data, estimator = "ML")
19 summary(fit1)
20
21
22
23 # 2. efficient method — using lower()/upper() modifiers
24 model2 <- '
25     Y == y1 + y2 + y3
26     X == x1 + upper(5)*x2 + x3
27     Y ~ X
28     x2 ~~ lower(0)*x2
29     '
30
31 # fit sem using nlminb()
32 fit2 <- sem(model2, data = Data, estimator = "ML")
33 summary(fit2)

```

Appendix E

Manual bounds: lavaan output

```

1  lavaan 0.6-9 ended normally after 37 iterations
2
3  Estimator                      ML
4  Optimization method            NLMINB
5  Number of model parameters     13
6
7  Number of observations         20
8
9  Model Test User Model:
10
11  Test statistic                 7.826
12  Degrees of freedom             8
13  P-value (Chi-square)          0.451
14
15  Parameter Estimates:
16
17  Standard errors                Standard
18  Information                    Expected
19  Information saturated (h1) model Structured
20
21  Latent Variables:
22      Estimate Std. Err z-value P(>|z|)
23  Y =~
24    y1          1.000
25    y2          1.410    1.056    1.335    0.182
26    y3          0.927    0.615    1.507    0.132
27  X =~
28    x1          1.000
29    x2          1.711    0.897    1.907    0.056
30    x3          0.511    0.305    1.675    0.094
31
32  Regressions:
33      Estimate Std. Err z-value P(>|z|)
34  Y ~
35    X          -0.035    0.230   -0.153    0.878
36
37  Variances:
38      Estimate Std. Err z-value P(>|z|)
39    .x2 (lb)    0.000    1.151    0.000
40    .y1          1.583    0.642    2.464    0.014
41    .y2          0.897    0.848    1.058    0.290
42    .y3          1.070    0.484    2.213    0.027
43    .x1          0.606    0.437    1.385    0.166
44    .x3          1.342    0.437    3.074    0.002
45    .Y          0.572    0.612    0.935    0.350
46    X          0.807    0.563    1.432    0.152

```

Appendix F

R code: lavaan input with automatic bounds

```

1 # 1. fit the model with standard bounds
2 model <- '
3   Y =~ y1 + y2 + y3
4   X =~ x1 + x2 + x3
5   Y ~ X
6 '
7
8 # fit sem using ML with standard bounds
9 fit.bounds.standard <-
10   sem(model, data = Data, estimator = "ML",
11     optim.bounds = list(lower = c("ov.var", "lv.var", "loadings"),
12       upper = c("ov.var", "lv.var", "loadings"),
13       min.reliability.marker = 0.1,
14       min.var.lv.endo = 0.005))
15 summary(fit.bounds)
16
17
18 # 2. fit the model with widened bounds
19 fit.bounds.wide <-
20   sem(model, data = Data, estimator = "ML",
21     optim.bounds = list(lower = c("ov.var", "lv.var", "loadings"),
22       upper = c("ov.var", "lv.var", "loadings"),
23       lower.factor = c(1.05, 1.0, 1.1),
24       upper.factor = c(1.20, 1.3, 1.1),
25       min.reliability.marker = 0.1,
26       min.var.lv.endo = 0.005))
27 summary(fit.bounds2)

```

Appendix G

Parameter table with automatic bounds

```

1 > parTable(fit.bounds.standard)[,c("lhs", "op", "rhs", "free",
2   "lower", "upper", "est")]
3
4   lhs op rhs free lower upper est
5 1   Y == y1  0  1.000 1.000 1.000
6 2   Y == y2  1 -3.074 3.074 1.410
7 3   Y == y3  2 -2.692 2.692 0.927
8 4   X == x1  0  1.000 1.000 1.000
9 5   X == x2  3 -4.089 4.089 1.711
10 6   X == x3  4 -3.315 3.315 0.511
11 7   Y ~ X  5  -Inf  Inf -0.035
12 8  y1 ~ y1  6  0.000 1.941 1.583
13 9  y2 ~ y2  7  0.000 2.037 0.897
14 10 y3 ~ y3  8  0.000 1.563 1.070
15 11 x1 ~ x1  9  0.000 1.271 0.606
16 12 x2 ~ x2 10  0.000 2.362 0.000
17 13 x3 ~ x3 11  0.000 1.552 1.342
18 14 Y ~ Y 12  0.005 2.156 0.572
19 15 X ~ X 13  0.141 1.413 0.807
20
21 > parTable(fit.bounds.wide)[,c("lhs", "op", "rhs", "free",
22   "lower", "upper", "est")]
23
24   lhs op rhs free lower upper est
25 1   Y == y1  0  1.000 1.000 1.000
26 2   Y == y2  1 -3.689 3.689 1.400
27 3   Y == y3  2 -3.231 3.231 0.938
28 4   X == x1  0  1.000 1.000 1.000
29 5   X == x2  3 -4.907 4.907 1.787
30 6   X == x3  4 -3.978 3.978 0.528
31 7   Y ~ X  5  -Inf  Inf -0.055
32 8  y1 ~ y1  6 -0.097 2.329 1.584
33 9  y2 ~ y2  7 -0.102 2.445 0.915
34 10 y3 ~ y3  8 -0.078 1.875 1.059
35 11 x1 ~ x1  9 -0.064 1.526 0.636
36 12 x2 ~ x2 10 -0.118 2.834 -0.118
37 13 x3 ~ x3 11 -0.078 1.863 1.336
38 14 Y ~ Y 12  0.005 2.802 0.570
39 15 X ~ X 13  0.141 1.794 0.777

```

Appendix H

R code: using wide bounds and the bootstrap

```

1 # fit the model with wide bounds + bootstrap
2 set.seed(1234)
3 fit.bootstrap <- sem(model, data = Data, estimator = "ML",
4   bounds = "wide", verbose = TRUE,
5   se = "bootstrap", bootstrap = 1000L)
6 summary(fit.bootstrap)

```

Appendix I

Bootstrap: lavaan output

```

1  lavaan 0.6-9 ended normally after 38 iterations
2
3  Estimator                      ML
4  Optimization method            NLMINB
5  Number of model parameters      13
6
7  Number of observations          20
8
9  Model Test User Model:
10
11  Test statistic                  6.789
12  Degrees of freedom              8
13  P-value (Chi-square)           0.560
14
15  Parameter Estimates:
16
17  Standard errors                 Bootstrap
18  Number of requested bootstrap draws 1000
19  Number of successful bootstrap draws 1000
20
21  Latent Variables:
22      Estimate  Std. Err  z-value  P(>|z|)
23  Y =~
24    y1          1.000
25    y2          1.392    0.972    1.432    0.152
26    y3          0.977    0.960    1.017    0.309
27  X =~
28    x1          1.000
29    x2          2.023    0.629    3.216    0.001
30    x3          0.558    0.390    1.429    0.153
31
32  Regressions:
33      Estimate  Std. Err  z-value  P(>|z|)
34  Y ~
35    X          -0.104    0.443    -0.235    0.814
36
37  Variances:
38      Estimate  Std. Err  z-value  P(>|z|)
39    .y1          1.597    0.725    2.201    0.028
40    .y2          0.953    0.771    1.236    0.216
41    .y3          1.029    0.555    1.853    0.064
42    .x1          0.715    0.298    2.399    0.016
43    .x2          (lb) -0.472    0.488    -0.968
44    .x3          1.335    0.347    3.852    0.000
45    .Y          0.552    0.679    0.814    0.416
46    X          0.698    0.438    1.595    0.111

```